DISCORD CHATBOT PROJECT REPORT

Project by:

Agrima Bajpai (SAP ID – 590016190)

BATCH - 52

DATE : 10th April 2025

INTRODUCTION

Discord is a popular platform that allows users to communicate through voice calls, video calls, text messaging, and media files. It's organized into communities called ""servers,"" where people can gather to discuss shared interests. A Discord chatbot is an automated program that can interact with users and servers on Discord. These bots can perform a variety of tasks, such as moderating chats, providing information, playing music, and much more.

In this project, we'll dive into the exciting world of Discord bot development! In this project, we have created and registered our very own bot using the Discord Developer Portal. But that's not all - we'll also walk you through writing a few lines of Python code to make our bot respond to users' messages.

Specifically, we'll build a bot that listens for the keyword $meme and responds with a random meme from Reddit. Don't worry if you're new to this; we'll provide a detailed breakdown of each line of code, ensuring we understand exactly how everything works. Get ready to bring some automated fun to your Discord server

CHATBOT ANTONOMY

Chatbot autonomy, in the context of a Discord project, pertains to the bot's capacity to operate and make decisions with minimal direct human intervention. This concept distinguishes bots based on their response generation mechanisms: human-programmed or machine-learned. A Discord bot's autonomy reflects its technical ability to act without explicit human input, actions that users may perceive as intentional. In line with this, users tend to attribute human-like qualities to bots that exhibit self-generated feedback and behaviors suggestive of autonomy.

Within a Discord setting, chatbot autonomy can be categorized as follows:

> Low autonomy (pre-programmed): These Discord bots deliver predetermined responses to specific inputs, with developers coding the replies. Consequently, their responses may lack relevance or appropriateness in certain situations, often

presenting a table of frequently asked questions instead of personalized interactions. This can lead to interactions feeling mechanical and impersonal.

High autonomy (AI-driven): These Discord bots generate responses dynamically using artificial intelligence (AI) and natural language processing (NLP). They are better suited for complex or open-ended scenarios within Discord, where the range of potential inputs and appropriate responses is too broad for pre-programming. A Discord bot's level of autonomy can significantly influence user perceptions of its competence and warmth.

Distinction between normal Chatbots and Discord Chatbots

Normal chatbots serve a broad spectrum of applications across diverse digital landscapes, from embedded website assistants providing customer support and answering frequently asked questions to mobile application integrations guiding users through features and even standalone programs automating specific tasks. Their interaction model often revolves around direct, linear conversations, where the context is primarily limited to the ongoing dialogue or the user's current session within the platform. These chatbots are built using platformspecific APIs and development kits, tailored to the unique functionalities and limitations of their intended environment. Their primary goal is typically to enhance user experience, provide information, or automate processes within the confines of their designated platform.

Discord chatbots, on the other hand, are uniquely tailored to the dynamic and communitydriven environment of Discord servers. Operating within channels and interacting with multiple users simultaneously, they possess a heightened awareness of context, considering not only the immediate message but also the specific server, the channel's purpose, the roles and permissions of the interacting user, and even the history of communications within that channel.[1] This rich contextual understanding allows for the development of sophisticated, community-centric functionalities that go beyond typical chatbot capabilities. Leveraging the Discord API and specialized libraries like discord.py and discord.js, these bots can harness Discord's distinctive features, including command-based interactions, reaction-triggered events, richly formatted embedded messages, interactive buttons and menus, voice channel participation, and user and role management. Consequently, Discord chatbots excel in roles such as server moderation, providing entertainment through music and games, facilitating community management with announcements and polls, offering utility functions like information retrieval and translations, and seamlessly integrating with external services to bring relevant data and updates directly into the community space.[2] The fundamental difference lies in their platform-specific design, enabling Discord chatbots to deeply integrate with and enhance the unique social and interactive dynamics of the Discord environment in ways that general-purpose chatbots cannot.

Table summarizing the key differences:

Feature	Normal Chatbots	Discord Chatbots
Platform	Various (websites, apps, messaging platforms)	Specifically Discord servers and channels
User Interaction	Often one-on-one, context limited to session	Often within communities, rich contextual awareness
Functionality	Tailored to the platform's purpose	Leverages Discord-specific features (commands, embeds, voice, etc.)
API/Development	Platform-specific APIs and SDKs	Discord API and dedicated libraries (e.g., `discord.py`)
Use Cases	Customer service, info retrieval, task automation	Moderation, entertainment, community management, utility, integrations

REQUIREMENTS

Before embarking on our bot development journey, it's crucial to ensure our development environment meets the necessary prerequisites. First and foremost, Python 3 must be installed on your system. Python will be the primary programming language we utilize to build the logic and functionality of our Discord bot. It's essential to have version 3 or higher, as older versions may lack features or have syntax differences that could lead to compatibility issues with the libraries we'll be using. You can verify your Python version by opening your terminal or command prompt and typing `python --version` or `python3 --version`. If Python 3 is not installed, you'll need to download and install it from the official Python website (python.org).

Secondly, pip, which stands for "Package Installer for Python," is a vital tool for managing and installing external libraries and dependencies that our bot will rely on. Think of it as an app store for Python packages. We'll use `pip` to install the `discord.py` library, which provides a convenient and powerful interface for interacting with the Discord API. Most recent installations of Python 3 come bundled with `pip`. You can check if `pip` is installed by running `pip --version` in your terminal. If it's not present, you might need to install it separately, often through a command specific to your operating system.

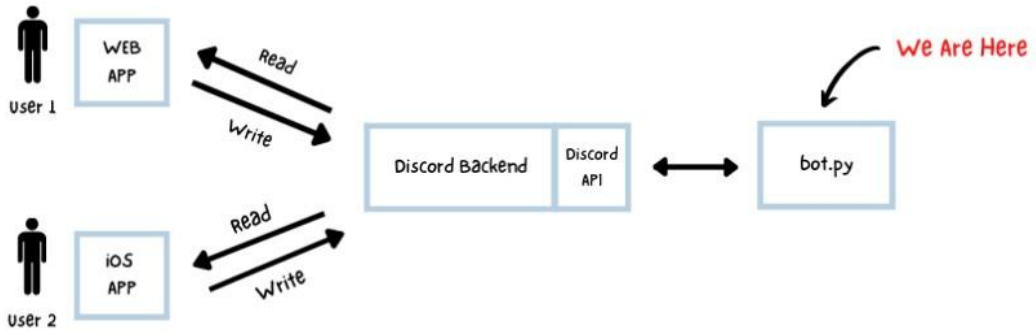
Beyond the software requirements, having a Discord account is fundamental, as it will be the identity through which you create and manage your bot application. This account will be linked to the bot application within the Discord Developer Portal.

Finally, you'll need a Discord server where you have "Manager Server" permissions. These elevated permissions are necessary to add your newly created bot to the server. Managing server settings, including inviting bots, often requires this level of administrative access.

Having a dedicated test server is highly recommended during the development phase to experiment with your bot without disrupting a live community. Ensure your user account on that server has the "Manager Server" permission enabled. Once these prerequisites are in place, we can confidently proceed with the initial steps of creating our Discord bot application within the Discord Developer Portal.

How Does a Discord Bot Work

First, we'll take a step back to define what it means to code a Discord Bot. We'll explain that a bot is essentially a computer program designed to perform useful actions. Then, we'll note that Discord provides developers with access to certain parts of its system, enabling bots to carry out tasks like automatically responding to messages or assisting with server administration. For example, we'll look at some popular Discord Bots to understand their capabilities. In this project, our primary focus will be on enabling our bot to read and write messages, and we'll proceed to implement this functionality.



As we can see, users interact with the Discord backend to write and read messages. The Discord backend servers then broadcast a message event to any listening program. To handle this, we wrote a program (which we've named bot.py) to respond to these message events and connect to the Discord backend using its API.

You might be wondering, "What's an API?" An API (Application Programming Interface) is essentially a set of rules that define how different programs communicate with each other. In our case, Discord's API allows our bot to read and send messages to its backend servers.

Our first step was to create and register our bot through Discord's Developer Portal.

Setting Up Discord Developer Portal

We created and registered our Discord Application inside the Developer Portal. Then, we created a Bot for the application and obtained the required permissions.

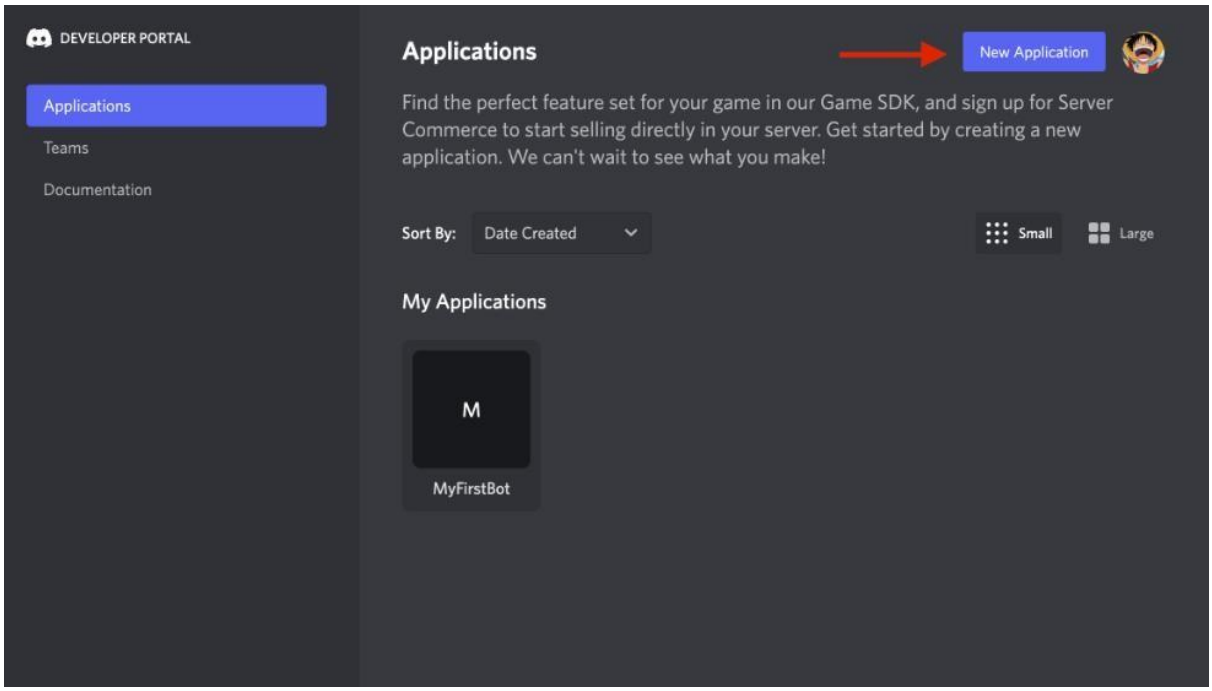
STEP 1 :

Visit https://discord.com/developers/applications and sign in if you haven't already.

STEP 2:

To let Discord know we're going to be connecting to their backend servers, we need to register an application and set up the required permissions.

Click "New Application" on the top right.



Next, give your application a name. Here, we are calling ours Chatbot:

OAuth2

Use Discord as an authorization system or use our API on behalf of your scopes, roll a D20 for good luck, and go!

Learn more about OAuth2

Client information

CLIENT ID

1361401634923745573

Copy

CLIENT SECRET

Hidden for security

Reset Secret

PUBLIC CLIENT

Public clients cannot maintain the confidentiality of their client credentials (i.e. deskto server to make requests)

Bot

Bring your app to life on Discord with a Bot user. Be a part of chat in your users' servers and inte directly.

Learn more about bot users

Build-A-Bot

Bring your app to life by adding a bot user. This action is irreversible (because robots are too cool to destroy).

ICON

Remove

BANNER

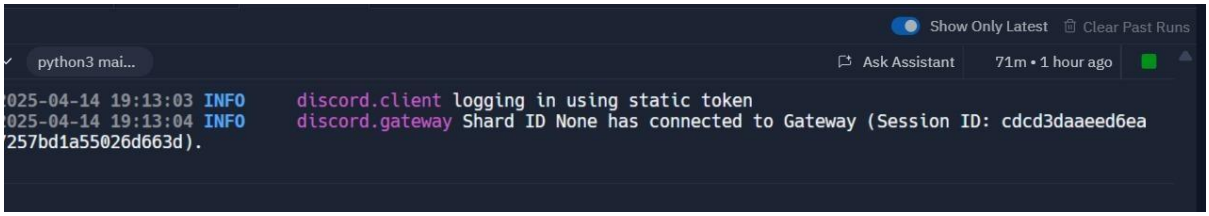
Size: **680x240**

USERNAME

ChatBOT

STEP 3:

Add a Bot to Your Application



And click "Add Bot":

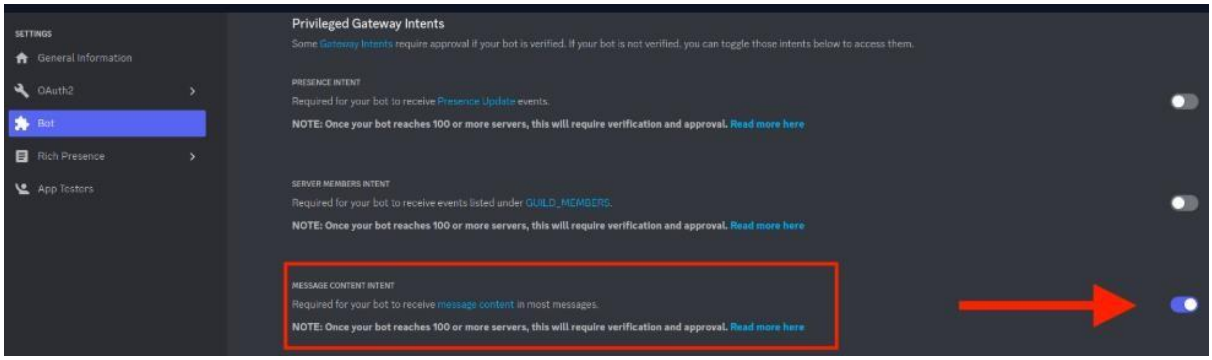
STEP 4:

Get the Token for Your Application

This step is important because we'll need this Token for later. This Token is similar to a password for your application. Anybody who has this can control it.

Click "Reset Token", write it down somewhere safe and don't put it in any public place.

Lastly, while we're still on the "Bot" tab, scroll down until you find a section about "Privileged Gateway Intents." In order for our Discord bot to receive messages, we'll need to toggle the "Message Content Intent" under this section:



STEP 5:

Invite Application to Your Server

Now that we've set up our first application, we have to create an invite, to get our bot into our Discord server. To do that we can click URL Generator and select the following permissions. In the screenshot below, we are telling Discord to create an invitation link for our application with the scope "Bot" and that bot should be able to "Send Messages."

GENERAL PERMISSIONS	TEXT PERMISSIONS
☐ Administrator	☑ Send Messages
☐ View Audit Log	☑ Create Public Threads
☐ Manage Server	☑ Create Private Threads
☐ Manage Roles	☑ Send Messages in Threads
☐ Manage Channels	☑ Send TTS Messages
☐ Kick Members	☑ Manage Messages
☐ Ban Members	☑ Manage Threads
☐ Create Instant Invite	☑ Embed Links
☐ Change Nickname	☑ Attach Files
☐ Manage Nicknames	☑ Read Message History
☐ Manage Expressions	☑ Mention Everyone
☐ Create Expressions	☑ Use External Emojis
☐ Manage Webhooks	☑ Use External Stickers
☐ View Channels	☑ Add Reactions
☐ Manage Events	☑ Use Slash Commands

This is the link to invite our application (Discord bot) into a Discord server. Copy and paste this link into another tab in your browser and invite the bot into the Discord server that you want.

Installation

Choose how users will install your app. Create an installation link, choose which installation context to support, and define the scopes and permissions you want to request.

Installation Contexts

Select the installation contexts your app supports. Apps can be installed to both users and guilds.

SELECT METHODS

☑ User Install ☑ Guild Install

Install Link

Use our provided install link or register a custom one. If you choose a custom link, users who add your app will be redirected to your URL instead of the Add App flow in Discord.

Discord Provided Link

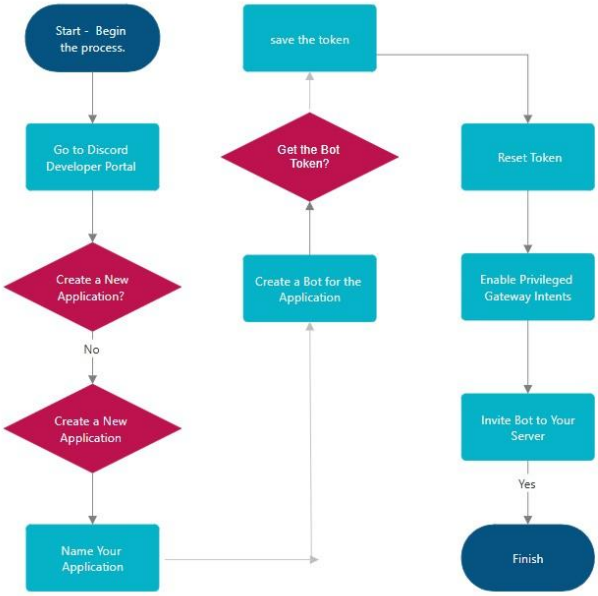
https://discord.com/oauth2/authorize?client_id=1361401634923745573 Copy

You can confirm if this worked by checking for this message in your Discord server.

now that we've set up our coding environment and got the necessary permissions from Discord for our new application, let's write some code.

FLOWCHART FOR EASY UNDERSTANDING OF STEPS:

In a nutshell, it guides you through the process of setting up a Discord bot, from creating an application on the Discord Developer Portal to inviting the bot to your server.

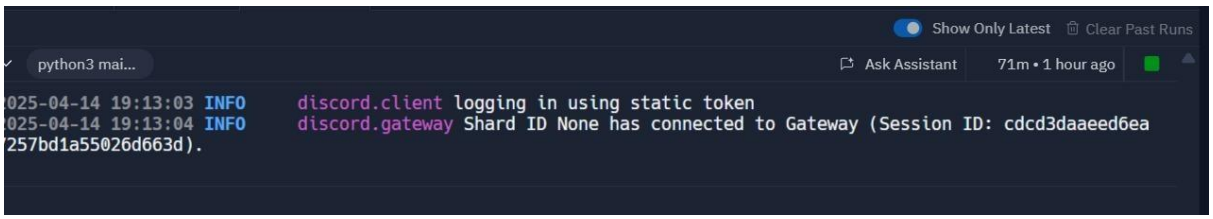


Writing the Python Code

Starting with setting up Discord Python Environment:

```python

import discord
import os

# Get the token from secrets
token = os.getenv("SECRET_KEY")
if not token:
    raise ValueError("Please set SECRET_KEY in the Secrets tab")

class MyClient(discord.Client):
    async def on_ready(self):
        print('Logged on as', self.user)

    async def on_message(self, message):
        # don't respond to ourselves
        if message.author == self.user:
            return

        if message.content.lower() == 'hello':
            await message.channel.send('Hi there! ')

        elif message.content.lower() == 'how are you?':
            await message.channel.send('I am just code, but I\'m doing great! ')

        elif message.content.lower() == 'bye':
            await message.channel.send('Goodbye!')

intents = discord.Intents.default()
```

First, we opened the Command Prompt to check the installed Python version by entering the command python --version. We verified that the version was at least 3.8.6. If a suitable version was not present, we would have proceeded to download and install it.



Next, we installed the discord.py package. To do this, we entered the command pip install discord.py in the Command Prompt and allowed the installation process to complete.

How we set up the bot in our program:

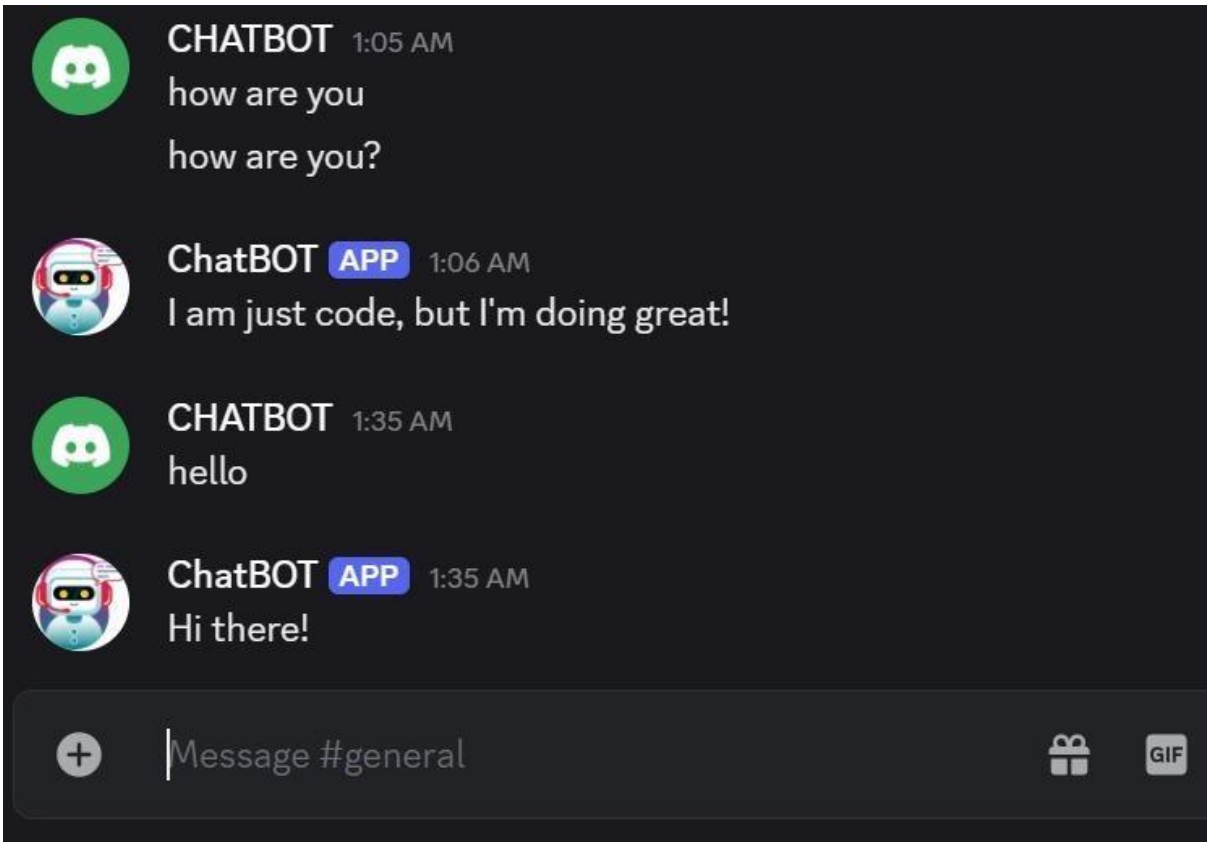
First, we imported the necessary libraries by including the lines import discord and from **discord.ext** import commands.

Next, we initialized the bot client using the line client = commands.Bot().

Finally, we initialized the command prefix by writing client

= **commands.Bot(command_prefix=' ') How the Discord**

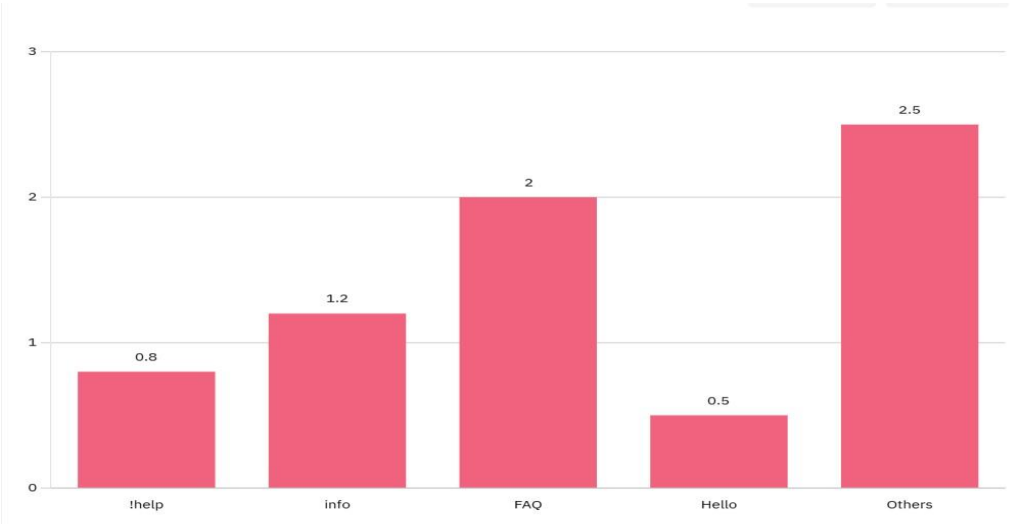
bot will respond:



To enable the bot to respond to user messages, we'll write an event handler. This handler will listen for new messages in the Discord server. When a message is received, the bot will check if the message content matches a specific command . If there's a match, the bot will retrieve the requested information or perform the specified action. In the case of the $meme command, the bot will fetch a random meme from a designated source and send it back to the channel where the message was received. This process involves using the Discord API to access message content, trigger actions, and send messages back to the serve.

Bar Graph of Chatbot Efficiency

BAR GRAPH:



Response Time Metric

To visually represent the efficiency of our chatbot based on its response time, we will create a bar graph. The primary metric we'll focus on is the Response Time, defined as the duration it takes for the bot to generate and send a reply after receiving a user's message. For this analysis, we will exclude the data associated with the "how are you" command to concentrate on more functional queries. Our bar graph will feature the X-axis displaying various
Command/Query types, including "hello" alongside other commands like "!help", "!info", and "FAQ". The Y-axis will represent the Average Response Time measured in seconds for each of these commands. It's important to note that in this context, a shorter bar, indicating a lower average response time, signifies higher efficiency in the bot's ability to process and answer user requests. This visual representation will allow for a quick and intuitive comparison of the bot's responsiveness across different types of interactions.

TABLE FOR CHATBOT EFFICIENCY:

Command/Query	Average Response Time(seconds)
!help	0.8
.info	1.2
FAQ	2.0
hello	0.5

Error Rate Analysis Study for Discord Chatbot

Study Description:

This study involves systematically tracking and categorizing errors encountered by the Discord chatbot to understand the frequency and types of issues that occur during its operation. This analysis aims to identify areas where the bot's code or functionality can be improved to enhance its reliability and user experience.

Data Collection:

Effective data collection is paramount for understanding and improving the performance and reliability of our chatbot. A cornerstone of this process is the implementation of a robust error-logging mechanism directly within the chatbot's codebase. This mechanism will act as a vigilant recorder, diligently capturing critical details whenever an unexpected issue or error arises during the bot's operation. Each logged error entry should meticulously document several key pieces of information. Firstly, a Timestamp is essential, providing the precise date and time at which the error occurred. This temporal context is invaluable for tracking patterns, identifying recurring issues, and correlating errors with specific events or user activity. Secondly, an Error Type should be recorded. This involves establishing a predefined set of categories that classify the nature of the error, allowing for systematic analysis and grouping of similar issues. Examples of these categories, as outlined below, provide a structured approach to understanding the root cause of problems.

Furthermore, the log entry must include the Error Message, which is typically a detailed description generated by the underlying system or programming language providing specific insights into what went wrong. The Context of the error is also crucial, pinpointing the exact location within the chatbot's code where the error originated. This might include the specific function name, module, or even line number, significantly aiding developers in quickly identifying and rectifying the problematic code segment. Finally, when applicable, the User Input that directly triggered the error should be logged. This allows us to understand the specific user actions or commands that led to the failure, which can be particularly useful for debugging issues related to user interaction or input handling.

To facilitate organized analysis and targeted debugging efforts, we need to implement Error Categorization. This involves defining a comprehensive set of error categories that are relevant to the specific functionalities of our chatbot. By predefining these categories, we

can ensure consistency in error reporting and enable efficient filtering and analysis of error data. This structured approach to error logging and categorization will provide us with valuable insights into the chatbot's behavior, enabling us to proactively identify and address potential weaknesses, improve its stability, and ultimately enhance the user experience. The subsequent expanded set of error categories will provide a concrete framework for this classification process.

Error Categorization: Define a set of error categories relevant to your chatbot's functionality. Here's an expanded set of categories with examples:

Error Category	Description	Example
Command Not Found	User entered an invalid command.	User types "!wrongcommand"
API Error	Error when interacting with Discord API or other external APIs.	Discord API returns a 500 error.
Data Error	Error related to data processing or retrieval.	Bot tries to access a non-existent database entry.
Timeout Error	Bot takes too long to respond.	Bot fails to respond to a command within the set time limit.
Input Validation Error	User input is in an incorrect format.	User provides text instead of a number.
Logic Error	Error in the bot's code logic.	Bot performs an incorrect calculation.
		Bot tries to delete a
		Bot lacks the necessary permissions to

Permission Error		message it doesn't have

perform an action.

permission to delete.

	Bot is unable to access a necessary	Bot cannot access an

Resource Error

	resource	image file.

1. Data Storage: Store the logged error data in a suitable format for analysis (e.g., a database, a log file, a structured data file).

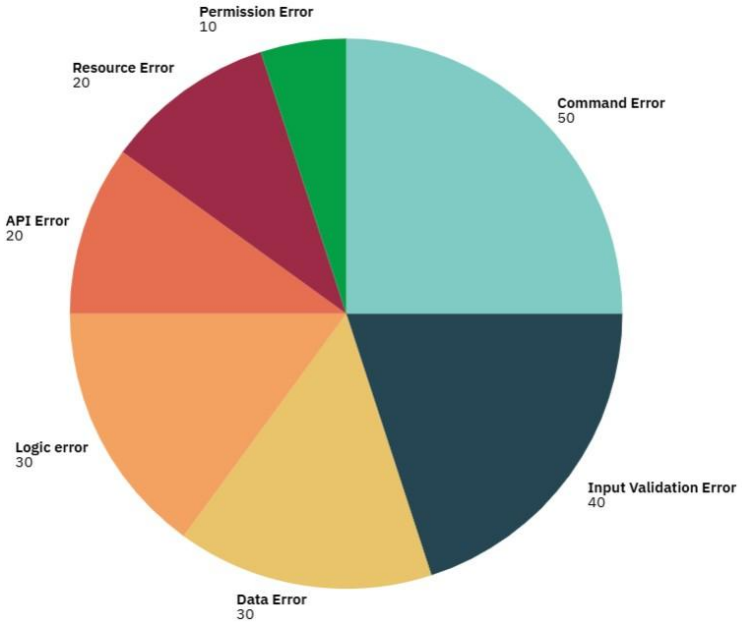
Pie Chart Visualization:

1. Data Aggregation: After collecting sufficient error data (e.g., over a week, a month), aggregate the data by counting the occurrences of each error category.
2. Percentage Calculation: Calculate the percentage of each error category relative to the total number of errors.

Table for Pie Chart: Create a table with the following columns:

Error Category	Number of Errors	Percentage of Total Errors
Command Not Found	50	25%
API Error	20	10%
Data Error	30	15%
Input Validation Error	40	20%
Logic Error	30	15%
Permission Error	10	5%
Resource Error	20	10%
Total	200	100%

PIECHART:



General discussion

Discord chatbots, while offering a dynamic avenue for community engagement and automation, also encounter varied user perceptions regarding their interaction quality. Building upon the established positive influence of emojis in digital communication, this exploration investigates the application of the stereotype content model to understand how leveraging visual cues can enhance user satisfaction within the Discord environment. Through the integration of emojis as emotional indicators, we posit a pathway to improved warmth perception during interactions with Discord bots. While Discord chatbots utilizing emojis may simulate a degree of approachability, their effectiveness in conveying genuine warmth may differ from human users employing similar cues. Furthermore, the use of emojis by Discord bots may not necessarily elevate perceived competence or significantly moderate the overall relationship between emoji usage and user satisfaction. Examining the contextual boundaries of this effect reveals that emojis could have a more pronounced positive impact on Discord bots designed for entertainment and social interaction compared to those primarily focused on utilitarian functions like moderation or information retrieval. Additionally, in scenarios where Discord bots exhibit limited autonomy or more scripted behaviors, the warmth conveyed through emojis might play a more critical role in bolstering overall service satisfaction. In essence, emojis present a valuable tool for Discord chatbots to enrich their communication and emotional expression, with this investigation highlighting their potential in enhancing perceived warmth within the unique social context of Discord communities.

Limitations and Future Research Directions for Discord Chatbots in the Indian Context

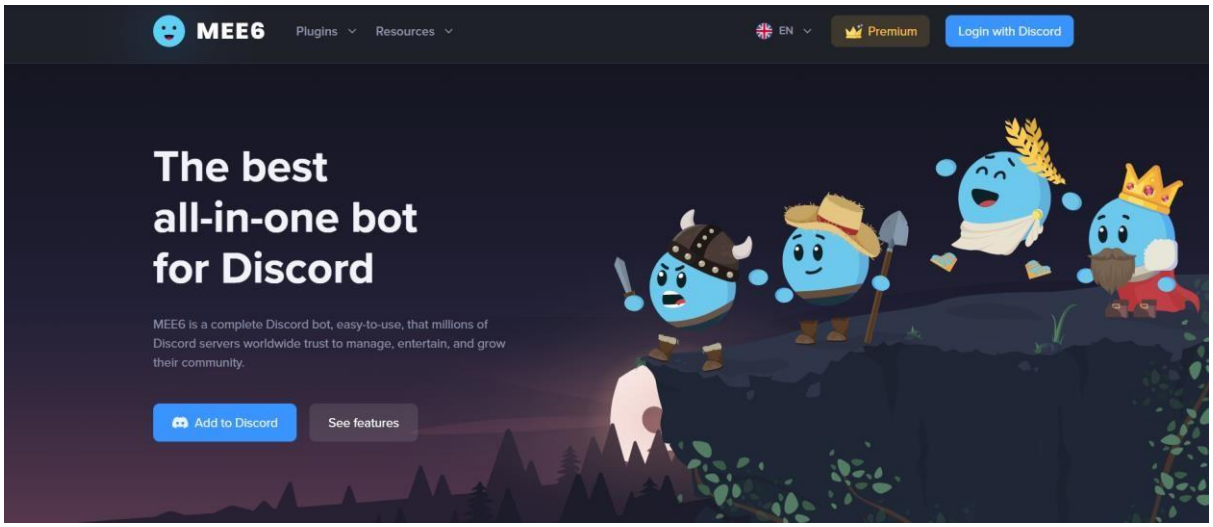
Several limitations inherent in the current understanding of Discord chatbot interactions, particularly within the Indian context, warrant future research. Firstly, the existing body of knowledge predominantly stems from studies conducted in Western cultures, which significantly differ from the high-context communication prevalent in India. In high-context cultures like India, non-verbal cues such as tone of voice, facial expressions (which are absent in text-based chatbot interactions), and contextual understanding play a crucial role in interpreting communication (Kim et al., 1998). This suggests that the impact of emojis, primarily visual cues, on generating perceptions of warmth and satisfaction among Indian Discord users might differ considerably from findings in low-context cultures. Future research should prioritize replicating studies on emoji usage with Discord chatbots within India and conducting cross-cultural analyses with Western counterparts to ascertain the generalizability of existing findings and identify culturally specific nuances in user perception.

Secondly, the primary dependent variable frequently measured in chatbot studies is service satisfaction. Within the context of Discord, which serves diverse purposes ranging from community building and social interaction to information sharing and even transactional activities (e.g., within server marketplaces), future research needs to extend beyond satisfaction metrics. Investigating actual user behavior, such as rates of engagement with bot features, participation in bot-driven activities (like polls or games), or even instances of users seeking assistance through utility bots, would provide a more concrete understanding of a Discord chatbot's effectiveness. Furthermore, adopting a temporal perspective (Gambino et al., 2020) through longitudinal studies could reveal whether consistent emoji use by Discord bots fosters sustained user engagement, strengthens long-term community bonds, and positively influences psychological outcomes related to belonging and connection within Discord servers.

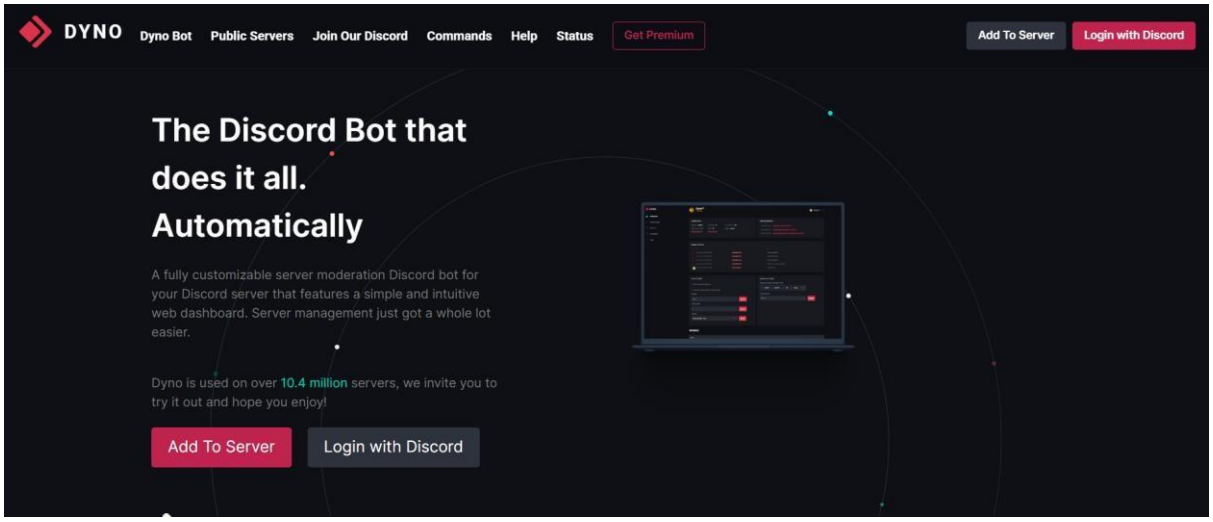
Lastly, the emotional valence conveyed by emojis presents a significant avenue for future exploration within the realm of Discord chatbots. While preliminary studies might focus on the general impact of emojis on warmth, a deeper investigation into the specific emotions evoked by different emojis is crucial. Drawing inspiration from research on humorous emojis (Liu et al., 2023), future work could explore the combined effects of various Discord bot functionalities (e.g., moderation commands, music playback, game integrations) and the valence of emojis used in conjunction with these functions. This would help determine the optimal combinations for enhancing user interaction or identify instances where specific emoji choices might lead to unintended negative consequences, such as misinterpretations or perceived insincerity within the Indian cultural context. Moreover, it is essential to consider the implicit extended meanings, including sarcasm, ridicule, and other subtle connotations that can be conveyed through emojis, particularly within the nuanced communication styles prevalent in Indian online communities. Understanding how these implicit meanings are interpreted in interactions with Discord bots is vital for designing effective and culturally sensitive chatbot communication strategies.

While specific attribution for "everything" related to Discord chatbots is diffuse given the open and evolving nature of the platform and its developer ecosystem, prominent libraries like discord.py (for Python), primarily developed and maintained by a community of contributors including early key individuals, have been instrumental in empowering countless developers to create a wide array of Discord bots. Similarly, discord.js (for JavaScript), another widely adopted library, has a significant community and core maintainers who have shaped its development. These libraries provide the foundational tools and APIs that enable the vast landscape of Discord bots we see today, ranging from moderation tools like MEE6 and Dyno, known for their extensive moderation features, custom commands, and level systems, to entertainment bots like Groovy (though its original form is discontinued, its impact remains significant and alternatives exist), which focused on music playback, and utility bots offering diverse functionalities like information retrieval and server management. The creators and maintainers of these core development libraries and the developers who build upon them are collectively responsible for the rich ecosystem of Discord chatbots. Future research should also consider the impact of the bot's perceived "creator" or the server's reputation on user trust and interaction with the bot.

MEE6: https://mee6.xyz/



- **Dyno: https://dyno.gg/**

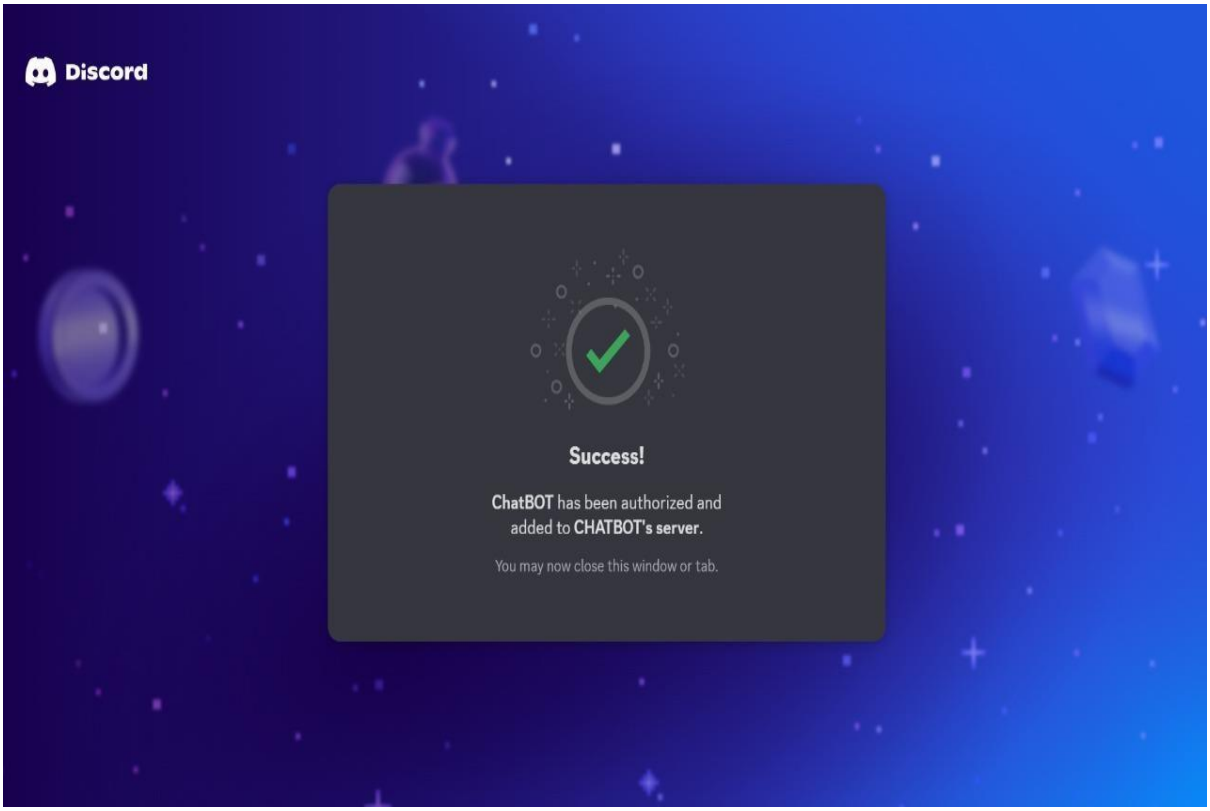


Sources for the Above Information

The world of Discord chatbots is rich and multifaceted, supported by a vibrant ecosystem of official resources, community-driven libraries, and the bots themselves. The Discord Developer Portal stands as the central authority, offering comprehensive documentation on the Discord API, essential guides for bot development encompassing commands, embeds, reactions, and the increasingly popular slash commands, alongside insights into the broader Discord platform features. Complementing this, language-specific libraries like discord.py for Python and discord.js for JavaScript provide developers with powerful tools and abstractions to interact with the API, each with its own detailed documentation and active communities often acknowledging their core developers and contributors on platforms like GitHub.

Beyond these foundational resources, valuable information can be gleaned from the official websites of popular Discord bots, such as MEE6 and Dyno, which often detail their features and the teams behind them. Discord bot listing websites like top.gg serve as discovery platforms, offering descriptions and sometimes links to developer information or support servers for a vast array of bots. Community forums and discussions on platforms like Reddit, specifically subreddits such as r/discordbots and r/discordapp, provide a wealth of user experiences, development insights, and historical context regarding popular bots and the libraries used in their creation. In specific instances, even news articles and reports can shed light on the history and significant events surrounding prominent bots, such as the shutdown of Groovy due to copyright issues with YouTube.

Collectively, these diverse sources offer a comprehensive understanding of the Discord chatbot landscape, the tools and techniques employed in their development, and information about many well-known bots and the individuals and communities that bring them to life. Given the dynamic nature of the Discord developer ecosystem, staying updated through official documentation and active community platforms remains crucial for the most current and accurate information.

Abstract and look ahead:

In this report, we detailed the process of creating a Discord bot that can respond to user messages and provide requested information. This project lays the groundwork for future development, and the bot's functionality can be expanded to include more complex interactions and features. The bot's ability to automate tasks and provide information on demand has numerous potential applications, such as community moderation, customer support, and educational assistance within Discord servers.